UNIDAD 7 · DEL CRUD AL E-COMMERCE · CLASE 2

El backend de la tienda

Catálogo, usuarios y compras: completamos los cinco huecos de la plantilla

Database | Auth | Middleware

Diseño Backend · 5.º año · Prof. Julia Panei

DÓNDE ESTAMOS

Los cinco huecos de hoy

1	`obtenerProductos`	`models/productosModel.js`	el catálogo
2	`registrar`	`models/authModel.js`	crear usuarios
3	`iniciarSesion`	`models/authModel.js`	el token
4	`verificarUsuario`	`middlewares/verificarUsuario.js`	¿quién compra?
5	`crearPedido`	`models/pedidosModel.js`	la compra

Los controllers y las rutas ya están hechos: hoy se trabaja en models y en el middleware.

ANTES DE ARRANCAR

Cómo se completa un hueco

Así viene

```
async function obtenerProductos(categoriaId) {
  // TODO (Clase 2 · Paso 1)
  throw faltaImplementar(...);
}
```

Así queda

```
async function obtenerProductos(categoriaId) {
  let consulta = supabase
    .from('productos')
    ...
  return data;
}
```

1. Borrá las dos líneas del TODO. **2.** Escribí el código de la práctica. **3.** Reiniciá el servidor y probá esa ruta antes de pasar a la siguiente.

PASO 1

El catálogo: el JOIN sin escribir JOIN

```
async function obtenerProductos(categoriaId) {
  const columnas = 'id, nombre, descripcion, precio, stock,' +
                   ' imagen_url, categorias(nombre)';
  let consulta = supabase
    .from('productos')
    .select(columnas)
    .eq('activo', true)
    .order('nombre');

  if (categoriaId) {
    consulta = consulta.eq('categoria_id', categoriaId);
  }

  const { data, error } = await consulta;
  if (error) throw error;
  return data;
}
```

Lo que llega

```
{
  "id": 1,
  "nombre": "El Eternauta",
  "precio": 32000,
  "stock": 6,
  "imagen_url":
"https://...",
  "categorias": {
    "nombre": "Novelas"
  }
}
```

let, no const: la consulta se arma de a partes.

PASOS 2, 3 Y 4

Usuarios: Supabase Auth hace lo difícil

1 Registro

`POST /registro`

Auth crea el usuario y guarda la contraseña cifrada

Paso 2

2 Login

`POST /login`

Auth controla la contraseña y devuelve un token

Paso 3

3 Compra

`POST /pedidos`

el front manda el token y el middleware pregunta a Auth de quién es

Paso 4

El token es la pulsera del recital: te la dan al entrar (login) y la mostrás en cada puerta (cada compra). No hace falta volver a dar la contraseña.

PASO 2

registrar: dos tablas, un mismo id

```
async function registrar(email, password, nombre) {
  const { data, error } = await
supabase.auth.admin.createUser({
    email,
    password,
    email_confirm: true          // no pedimos confirmar por
mail
  });
  if (error) throw error;

  const { error: e2 } = await supabase.from('perfiles')
    .insert({ id: data.user.id, nombre });
  if (e2) throw e2;
  return data.user;
}
```

`auth.admin`

solo funciona con la clave secreta

`email_confirm: true`

lo da por confirmado: no hace falta configurar mails

`data.user.id`

el id que le dio Auth, que reusamos en perfiles

Para comprobarlo: Authentication → Users en el panel, y la tabla perfiles.

PASO 3

iniciarSesion y el cliente descartable

```
async function iniciarSesion(email, password) {
  const temporal = createClient(
    process.env.SUPABASE_URL,
    process.env.SUPABASE_KEY,
    { auth: { persistSession: false } }
  );
  const { data, error } =
    await temporal.auth.signInWithPassword({ email,
password });
  if (error) return null;           // datos incorrectos
  return data.session.access_token;
}
```

¿Por qué un cliente nuevo?

El cliente con el que se hace el login queda "logueado" como ese usuario. Si fuera el principal, todo el servidor pasaría a trabajar con los permisos de un cliente. El temporal se usa una vez y se descarta.

`return null` no es un error del servidor: el controller lo convierte en 401 "Email o contraseña incorrectos".

PASO 4

verificarUsuario: el control de la puerta

```
async function verificarUsuario(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.replace('Bearer ', '');
  if (!token) return res.status(401).send('Tenes que iniciar sesion');

  const { data, error } = await supabase.auth.getUser(token);
  if (error || !data.user) return res.status(401).send('Sesion invalida');

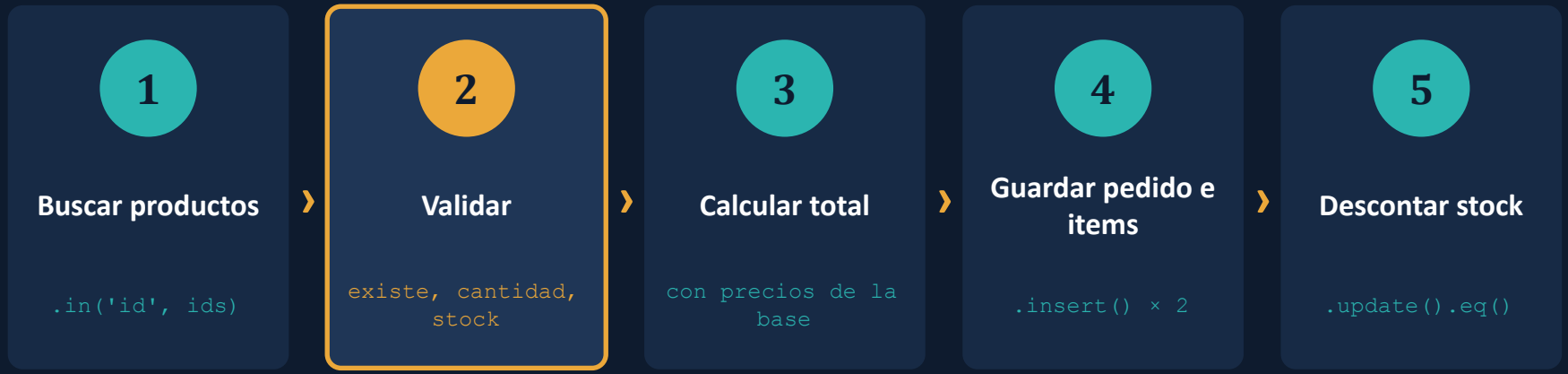
  req.usuario = data.user;     // desde aca, el controller sabe quien compra
  next();
}
```

```
router.post('/pedidos', verificarUsuario, pedidos.crearPedido);
```

401 no sabemos quién sos. **403** sabemos, pero no tenés permiso.

PASO 5

La compra: cinco cosas en orden



`.in('id', [1, 3])` trae varios productos en una sola consulta: es el WHERE id IN (1, 3) de SQL.

El usuario sale del token (req.usuario.id), nunca del body.

PASO 5 · PRIMERA PARTE

Buscar, validar y calcular

```
// 1. Buscamos todos los productos del carrito en una sola consulta
const ids = items.map(i => i.producto_id);
const { data: productos, error: e1 } = await supabase
  .from('productos').select('id, nombre, precio, stock')
  .in('id', ids).eq('activo', true);
if (e1) throw e1;

// 2 y 3. Validamos y calculamos con precios de la base
let total = 0;
const filas = [];
for (const item of items) {
  const p = productos.find(x => x.id === item.producto_id);
  if (!p) throw errorCliente('No existe el producto ' + item.producto_id);
  if (!Number.isInteger(item.cantidad) || item.cantidad < 1) {
    throw errorCliente('Cantidad invalida para ' + p.nombre);
  }
  if (p.stock < item.cantidad) throw errorCliente('Sin stock de ' + p.nombre);
  total += p.precio * item.cantidad;
  filas.push({ producto_id: p.id, cantidad: item.cantidad, precio_unitario: p.precio });
}
```

PASO 5 · SEGUNDA PARTE

Guardar y descontar

```
// 4. Guardamos el pedido y despues sus items
const { data: pedido, error: e2 } = await supabase
  .from('pedidos')
  .insert({ usuario_id: usuarioId, total })
  .select()
  .single();
if (e2) throw e2;

const conPedido = filas.map(f => ({ ...f, pedido_id: pedido.id }));
const { error: e3 } = await supabase.from('pedido_items').insert(conPedido);
if (e3) throw e3;

// 5. Descontamos el stock de cada producto
for (const f of filas) {
  const p = productos.find(x => x.id === f.producto_id);
  await supabase.from('productos')
    .update({ stock: p.stock - f.cantidad }).eq('id', p.id);
}
return pedido;
```

`pedido.id`

los items lo necesitan: por eso el pedido se guarda primero

`...f`

copia el item y le suma pedido_id

`.insert(lista)`

guarda todos los items juntos

CÓMO VIAJA EL 400

Un error que lleva su propio código

En el model (ya viene hecho)

```
function errorCliente(mensaje) {
  const e = new Error(mensaje);
  e.status = 400;
  return e;
}

throw errorCliente('Sin stock de Rayuela');
```

En el controller (ya viene hecho)

```
} catch (error) {
  if (error.status) {
    return res.status(error.status)
      .send(error.message);
  }
  res.status(500).send('Error...');
}
```

Con status: culpa del cliente, responde 400 con el mensaje. **Sin status:** falló la base, responde 500.

PASO 6

Siete pruebas en Thunder Client

#	Request	Body	Esperado
1	`GET /productos`	sin body	**200 con categorias**
2	`POST /registro`	email, password, nombre	**201**
3	`POST /login`	email, password	**200 y el token**
4	`POST /pedidos sin token`	items	**401**
5	`POST /pedidos con token`	items	**201 y el pedido**
6	`POST /pedidos con token`	cantidad mayor al stock	**400 sin stock**
7	`POST /login`	contraseña incorrecta	**401**

Después de la 5: un pedido nuevo, sus items con precio_unitario y el stock descontado en el Table Editor.

OJO CON ESTO

Los errores que más van a aparecer

! Siempre 401 aunque hiciste login

Falta "Bearer " o el token venció (dura una hora).

✓ Pestaña Auth → Bearer

! Registro: not allowed

Estás usando la clave pública: auth.admin pide la secreta.

✓ SUPABASE_KEY = secret key

! 'errorCliente' already been declared

Pegaste errorCliente otra vez: ya está en la plantilla.

✓ Pegá solo el cuerpo de crearPedido

! Total en 0 o en NaN

La cantidad llega como texto: "2" en vez de 2.

✓ "cantidad": 2, sin comillas

ANTES DE IRSE

El checklist de hoy

- [ ] **Catálogo**
  GET /productos devuelve tus productos con su categoría
- [ ] **Filtro**
  GET /productos?categoria=1 devuelve solo esa categoría
- [ ] **Registro**
  el usuario aparece en Authentication y en perfiles
- [ ] **Login**
  POST /login devuelve un token
- [ ] **Compra**
  con token: 201, items guardados y stock descontado
- [ ] **Controles**
  sin token 401, sin stock 400

En el navegador los productos todavía no aparecen: eso es el front, en la Clase 3.

PARA LLEVARSE

Tres ideas de hoy

1

Las contraseñas no son nuestras.

Auth las guarda cifradas; nosotros solo manejamos el token.

2

El token dice quién compra.

El middleware lo verifica y el usuario sale de req.usuario, nunca del body.

3

Primero se valida, después se escribe.

Si falta stock, la compra se corta antes de tocar la base.

Próxima clase: el front con fetch. Los productos aparecen en la página.